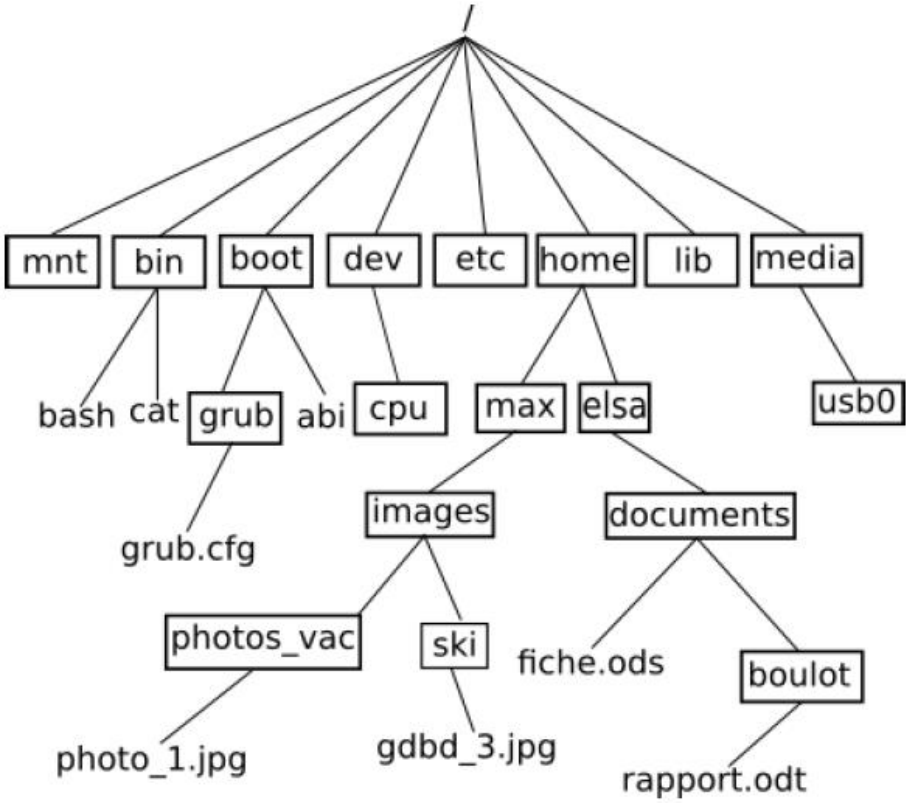
Structures de données arborescentes. Les arbres. Correction.

I) QU'EST-CE QU'UN ARBRE ?

1°) Notion d'arbre :

Les **structures de données** que nous avons étudiées jusqu'à présent (tableaux, listes, piles, files…) sont **linéaires**, dans la mesure où elles stockent les éléments les uns à la suite des autres « à la queue leu leu ». On peut représenter de telles structures d'une manière imagée comme des éléments placés sur une ligne à l'instar d'oiseaux posés sur un fil électrique.

Un **arbre** est une structure constituée de **nœuds**, qui peuvent avoir des **enfants** qui sont eux-mêmes des nœuds. Un arbre est donc une structure de données **arborescente**, ou **hiérarchique**. Les systèmes de fichiers dans les systèmes de type UNIX (Linux et Mac OS) ont par exemple une structure en arbre définissant une arborescence :



Système de fichier UNIX (arborescence)

Les arbres sont des structures mathématiques abstraites très utilisées en informatique. On les utilise notamment quand on a besoin d'une structure hiérarchisée des données : dans l'exemple ci-dessus le fichier grub.cfg ne se trouve pas au même niveau de l'arborescence que le fichier rapport.odt (le fichier grub.cfg se trouve "plus proche" de la racine / que le fichier rapport.odt). On ne pourrait pas avec une simple liste qui contiendrait les noms des fichiers et des répertoires, rendre compte de cette hiérarchie (plus ou moins "proche" de la racine).

Exemples de domaines dans lesquels on rencontre des arbres :

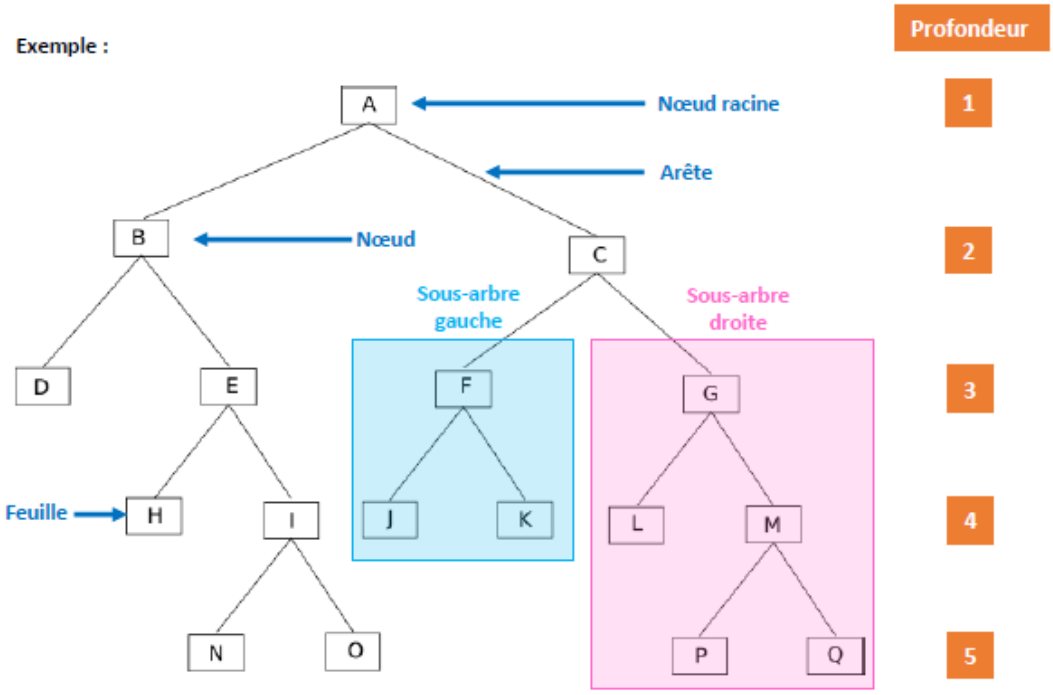
- Système de fichiers (arborescence répertoires / fichiers).
- Fonctionnement d'une interface graphique (Panels et widgets).
- Arbres généalogiques, arbre phylogénétique ...

2°) Notion d'arbre binaire :

Les arbres binaires sont des cas particuliers d'arbre : dans un arbre binaire, on a au maximum 2 branches qui partent d'un élément. Dans la suite de l'activité nous travaillerons principalement sur les arbres binaires.

■ **Un peu de vocabulaire**

En informatique, un **arbre binaire** est une **structure de données** qui peut se représenter sous la forme d'une **hiérarchie** dont chaque élément est appelé **nœud**, le **nœud initial** étant appelé **racine**. Dans un arbre binaire, chaque élément possède au plus **deux éléments fils** au **niveau inférieur**, habituellement appelés **gauche** et **droit**. Du point de vue de ces éléments fils, l'élément dont ils sont issus au niveau supérieur est appelé **père**.
Au niveau le plus élevé il y a donc un nœud racine. Au niveau directement inférieur, il y a au plus deux nœuds fils. En continuant à descendre aux niveaux inférieurs, on peut en avoir quatre, puis huit, seize, etc. c'est-à-dire la suite des puissances de deux. Un nœud n'ayant aucun fils est appelé **feuille**. Le nombre de niveaux total, autrement dit la distance entre la feuille la plus éloignée et la racine, est appelé **hauteur** de l'arbre.
Le niveau d'un nœud est appelé **profondeur**.
La **taille** d'un arbre est son nombre de nœuds.



■ **Quelques précisions concernant l'arbre binaire précédent :**

- Chaque élément de l'arbre est appelé **nœud** (par exemple : A, B, C, D, ..., P et Q sont des nœuds).
- Le nœud initial (A) est appelé **nœud racine** ou plus simplement **racine**.
- On appelle **arête** le segment qui relie 2 nœuds.
- Le nœud E et le nœud D sont les **fils** du nœud B. Réciproquement, on dira que le nœud B est le **père** des nœuds E et D.
- Dans un arbre binaire, un nœud possède au plus 2 fils.
- Un nœud n'ayant aucun fils est appelé **feuille** (exemples : D, H, N, O, J, K, L, P et Q sont des feuilles).
- À partir d'un nœud (qui n'est pas une feuille), on peut définir un **sous-arbre gauche** et un **sous-arbre droit** (exemple : à partir de C on va trouver un sous-arbre gauche composé des nœuds F, J et K et un sous-arbre droit composé des nœuds G, L, M, P et Q).
- La **profondeur** d'un nœud ou d'une feuille dans un arbre binaire est le nombre de nœuds du chemin qui va de la racine à ce nœud. La racine d'un arbre est à une profondeur 1, et la profondeur d'un nœud est égale à la profondeur de son prédécesseur plus 1. Si un nœud est à une profondeur p, tous ses successeurs sont à une profondeur p+1. **Exemples :** profondeur de B = 2 ; profondeur de I = 4 ; profondeur de P = 5.
- La hauteur d'un arbre est la profondeur maximale des nœuds de l'arbre. **Exemple :** la profondeur de P = 5, c'est un des nœuds les plus profond, donc la hauteur de l'arbre est de 5.

ATTENTION : on trouve aussi dans certains livres la profondeur de la racine égale à 0 (on trouve alors : profondeur de B = 1 ; profondeur de I = 3 ; profondeur de P = 4). Les 2 définitions sont valables, il faut juste préciser si vous considérez que la profondeur de la racine est de 1 ou de 0.
On appelle hauteur d'un arbre la profondeur maximale des nœuds de l'arbre. Exemple : la profondeur de P = 5, c'est un des nœuds les plus profonds, donc la hauteur de l'arbre est de 5.

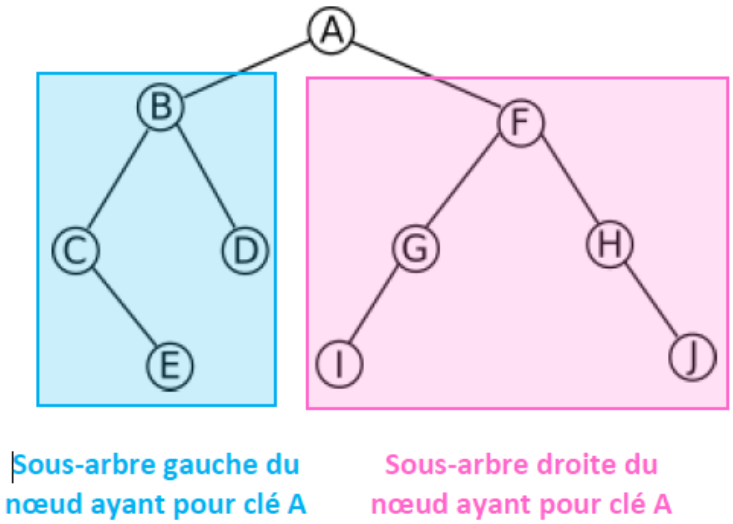
ATTENTION : comme on trouve 2 définitions pour la profondeur, on peut trouver 2 résultats différents pour la hauteur : si on considère la profondeur de la racine égale à 1, on aura bien une hauteur de 5, mais si l'on considère que la profondeur de la racine est de 0, on aura alors une hauteur de 4.
Cette année, sauf indication contraire, on considère que la hauteur de la racine est de 1.

Il est important de remarquer que l'on peut aussi voir les arbres comme des structures récursives : les fils d'un nœud sont des arbres (sous-arbre gauche et un sous-arbre droit dans le cas d'un arbre binaire), ces arbres sont eux-mêmes constitués d'arbres...

Remarque : Python ne propose pas de façon native l'implémentation des arbres binaires. Il faudra pour les modéliser faire appel à la programmation orientée objet (P.O.O.) comme nous allons le voir-ci après.

3°) Arbre binaire et clés :

À chaque nœud d'un arbre binaire, on peut associer une **clé** correspondant au nom du nœud et servant d'**étiquette**.



En partant du nœud racine dont la clé est A on peut distinguer :

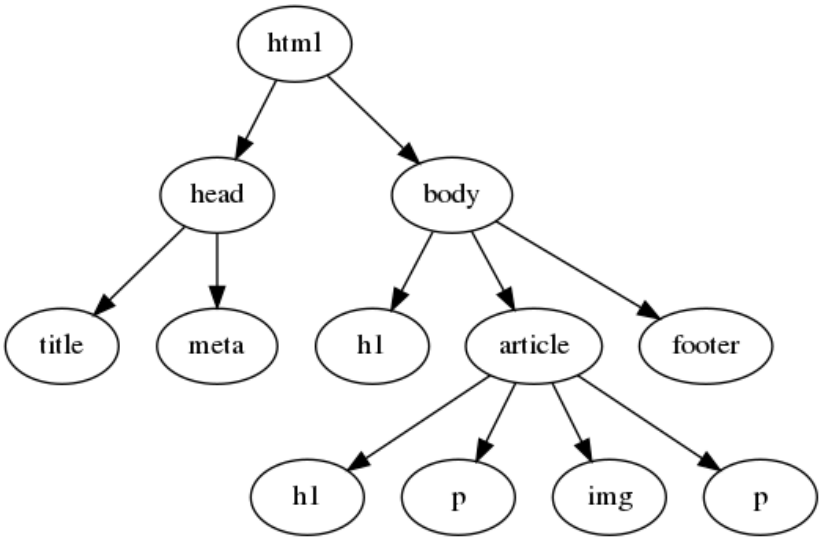
- le sous-arbre gauche composé du nœud ayant pour clé B, du nœud ayant pour clé C, du nœud ayant pour clé D et du nœud ayant pour clé E ;
- le sous-arbre droite est composé du nœud ayant pour clé F, du nœud ayant pour clé G, du nœud ayant pour clé H, du nœud ayant pour clé I et du nœud ayant pour clé J.

Un **arbre** (ou un **sous-arbre**) **vide** est noté **NIL** (abréviation du latin nihil qui veut dire rien).
Si on prend le nœud ayant pour clé G on a :

- le sous-arbre gauche est uniquement composé du nœud ayant pour clé I ;
- le sous-arbre droit est vide (NIL).

Il faut bien avoir à l'esprit qu'un sous-arbre (gauche ou droite) est un arbre (même s'il contient un seul nœud ou pas de nœud du tout (NIL)).

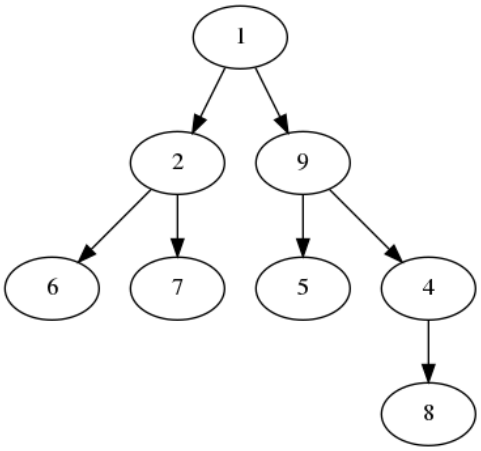
Exercice 1. Premier exemple : un arbre en html.
En voici un exemple :



a) Combien de feuilles possède-t-il ?
8.
b) Quelle est l'étiquette de cet arbre ?
'html'.
c) Quelle est sa taille ?
12.
d) Quelle est sa hauteur ?
4.
e) Quelle est la profondeur du nœud dont l'étiquette est article ?
3.
f) Combien d'enfants possède cet arbre ?
2 seulement : les arbres dont les étiquettes sont 'head' et 'body'.
g) Quel est le nœud qui possède le plus d'enfants ?
Celui dont l'étiquette est 'article'.

II) PARCOURIR UN ARBRE :

Réaliser un parcours d'un arbre, c'est visiter chacun de ses nœuds dans un certain ordre et leur appliquer un traitement.



- **Dans un parcours en largeur, les nœuds sont parcourus par profondeur croissante.** Voici un algorithme possible qui a la particularité d'utiliser une structure de file avec les méthodes enfiler et defiler :

```
ParcoursLargeur(Arbre A) {
  f = FileVide
  enfiler(Racine(A), f)
  Tant que (f != FileVide) {
    nœud = defiler(f)
    Visiter(nœud)                    //On choisit de faire une opération
    Si (gauche(nœud) != null) Alors
      enfiler(gauche(nœud), f)
```

```
    Si (droite(nœud) != null) Alors
        enfiler(droite(nœud), f)
  }
}
```

Avec l'exemple donné, les étiquettes seront traitées dans l'ordre suivant : 1 > 2 > 9 > 6 > 7 > 5 > 4 > 8.

- **Un parcours en profondeur préfixe est un parcours où on effectue le traitement de chaque nœud avant d'explorer le sous-arbre correspondant.**

```
def parcours_profondeur_prefixe(noeud):
  traitement(etiquette)
  pour chaque enfant de noeud:
    parcours_profondeur_prefixe(enfant)
```

Avec l'exemple donné, les étiquettes seront traitées dans l'ordre suivant : 1 > 2 > 6 > 7 > 9 > 5 > 4 > 8.

- **Un parcours en profondeur postfixe est un parcours où l'on effectue le traitement de chaque nœud après avoir exploré le sous-arbre correspondant.**

```
def parcours_profondeur_post(noeud):
  pour chaque enfant de noeud:
    parcours_profondeur_post(enfant)
  traitement(etiquette)
```

Avec l'exemple donné, les étiquettes seront traitées dans l'ordre suivant : 6 > 7 > 2 > 5 > 8 > 4 > 9 > 1.

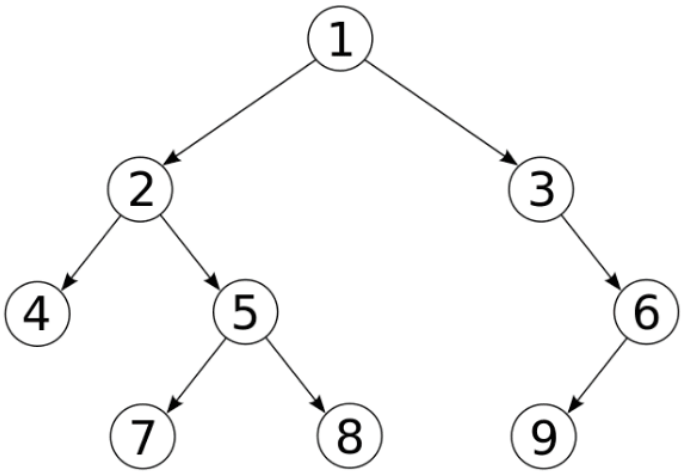
- **Dans un parcours en profondeur infixe (inorder traversal), chaque nœud est visité après son enfant gauche mais avant son enfant droit.** C'est une manière assez commune de parcourir un arbre binaire de recherche, car il donne les valeurs dans l'ordre croissant dans le cas d'un **arbre binaire de recherche**.

```
def visiterInfixe(Arbre A) :
  Si nonVide(gauche(A))
    visiterInfixe(gauche(A))
  visiter(A)
  Si nonVide(droite(A))
    visiterInfixe(droite(A))
```

Pour comprendre pourquoi cela est le cas, si *n* est un nœud dans un **arbre binaire de recherche** (*un arbre binaire de recherche est un arbre binaire dans lequel chaque nœud possède une clé, telle que chaque nœud du sous-arbre gauche ait une clé inférieure ou égale à celle du nœud considéré, et que chaque nœud du sous-arbre droit possède une clé supérieure ou égale à celle-ci — selon la mise en œuvre de l'ABR, on pourra interdire ou non des clés de valeur égale. Les nœuds que l'on ajoute deviennent des feuilles de l'arbre*), alors tous les éléments dans le sous-arbre de gauche du nœud *n* seront inférieurs à *n* et ceux dans le sous-arbre de droite seront supérieurs ou égaux à *n*. Ainsi, si nous visitons le sous-arbre de gauche dans l'ordre, de manière récursive, puis que nous visitons *n*, et que nous visitons le sous-arbre de droite, nous aurons visité l'ensemble du sous-arbre enraciné en *n* dans l'ordre.

Avec l'exemple donné, les étiquettes seront traitées dans l'ordre suivant : 6 > 2 > 7 > 1 > 5 > 9 > 8 > 4, si on considère que 8 est le sous-arbre gauche de 4, et 6 > 2 > 7 > 1 > 5 > 9 > 4 > 8, si on considère que 8 est le sous-arbre droit de 4.

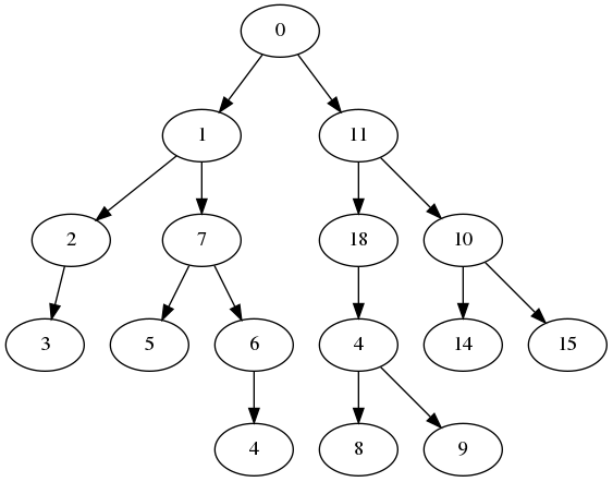
<u>En résumé, autre exemple</u> :



Dans cet arbre binaire,

- Rendu du parcours préfixe : 1, 2, 4, 5, 7, 8, 3, 6, 9
- Rendu du parcours postfixe : 4, 7, 8, 5, 2, 9, 6, 3, 1
- Rendu du parcours infixe : 4, 2, 7, 5, 8, 1, 3, 9, 6
- Rendu du parcours en largeur : 1, 2, 3, 4, 5, 6, 7, 8, 9

Exercice 2. Exemples de parcours.
Avec l'arbre ci-dessous :



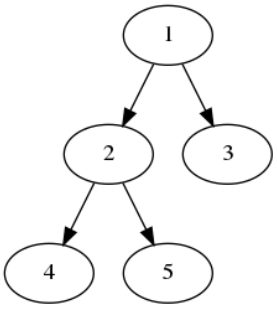
a) Dans quel ordre les étiquettes sont-elles rencontrées dans un parcours en largeur ?
0 - 1 - 11 - 2 - 7 - 18 - 10 - 3 - 5 - 6 - 4 - 14 - 15 - 4 - 8 - 9.
b) Dans quel ordre les étiquettes sont-elles traitées dans un parcours en profondeur postfixe ?
3 - 2 - 5 - 4 - 6 - 7 - 1 - 8 - 9 - 4 - 18 - 14 - 15 - 10 - 11 - 0.
c) Dans quel ordre les étiquettes sont-elles traitées dans un parcours en profondeur préfixe ?
0 - 1 - 2 - 3 - 7 - 5 - 6 - 4 - 11 - 18 - 4 - 8 - 9 - 10 - 14 - 15.
d) Dans quel ordre les étiquettes sont-elles traitées dans un parcours en profondeur infixe ?
3 - 2 - 1 - 5 - 7 - 4 - 6 - 0 - 8 - 4 - 9 - 18 - 11 - 14 - 10 - 15.
e) Quelle est l'étiquette de cet arbre ?
0.
f) Quelle est sa taille ?
16.
g) Quelle est sa hauteur ?
4.
h) Quelle est la profondeur du nœud dont l'étiquette est 10 ?
2.
i) Combien d'enfants possède cet arbre ?
2 : les arbres d'étiquettes '1' et '11'.
j) Quel est l'étiquette la plus grande ?
18.

III) DES ARBRES AVEC PYTHON (MAIS TOUJOURS SANS ORDINATEUR !) :

Exercice 3. Construction d'arbres.

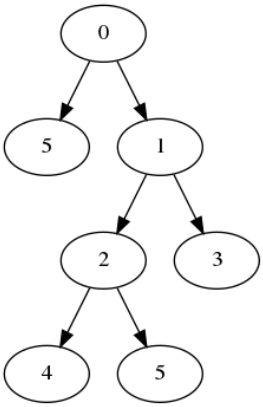
Voici un exemple de code permettant de construire l'arbre a1 représenté ci-contre.

```
a1 = Arbre(1)
b = Arbre(2)
c = Arbre(3)
d = Arbre(4)
a1.ajoute(b, c)
b.ajoute(d, Arbre(5))
a1.view()
```

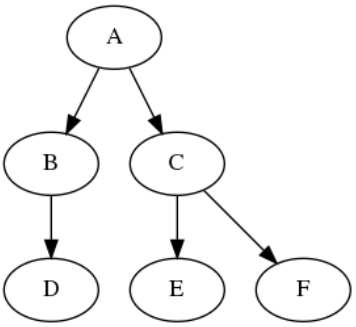


a) Dessiner l'arbre a2 construit par le code suivant :

```
a2 = Arbre(0)
a2.ajoute(Arbre(5), a1)
a2.view()
```



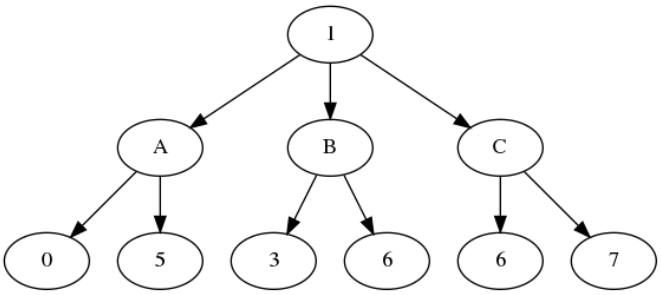
b) Donner le code permettant de construire l'arbre a3 suivant :




```
a3 = Arbre('A')
a3.ajoute(Arbre('B'), Arbre('C'))
a3.enfants[0].ajoute(Arbre('D'))
a3.enfants[1].ajoute(Arbre('E'), Arbre('F'))
a3.view()
```

c) Dessiner l'arbre a5 construit par le code suivant, sachant que arbre.enfants est la liste des enfants de arbre.

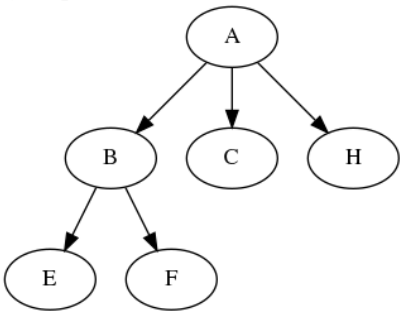
```
a4 = Arbre(1)
a4.ajoute(Arbre('A'), Arbre('B'), Arbre('C'))
for i in range (3):
    a4.enfants[i].ajoute(Arbre(3*i), Arbre(i+5))
a4.view()
```

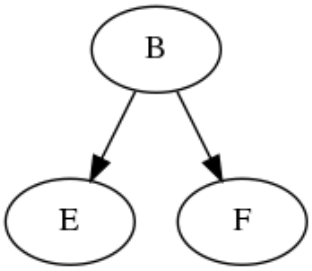


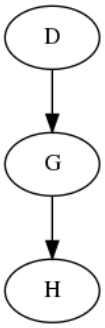
Exercice 4. Construction d'arbres.
Soit le code :

```
A = Arbre('A')
B = Arbre('B')
C = Arbre('C')
D = Arbre('D')
E = Arbre('E')
F = Arbre('F')
G = Arbre('G')
H = Arbre('H')
B.ajoute(E, F)
D.ajoute(G)
A.ajoute(B, C, H)
G.ajoute(H)
```

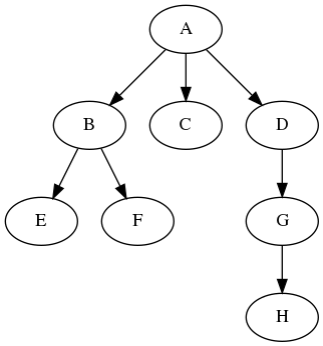
a) Représenter les arbres A, B et D.

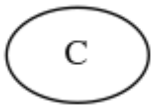






b) Représenter les arbres A et C après l'instruction : A.enfants[2] = D.





On admet que si T est un arbre :

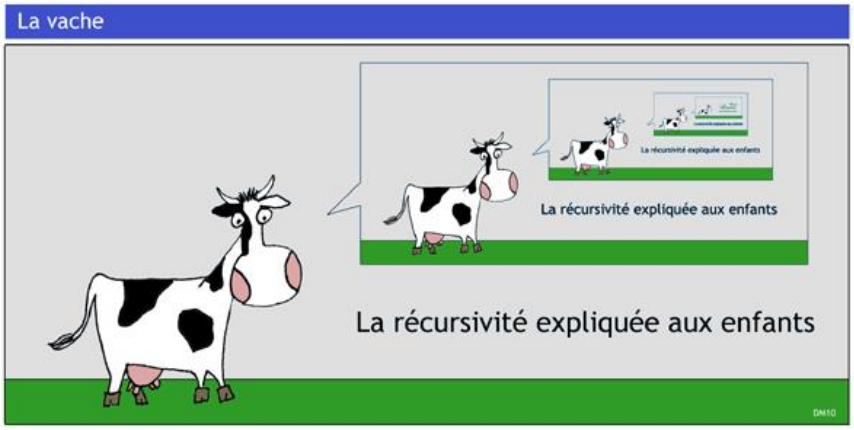
- T.enfants est la liste des enfants de T.
- T.label est l'étiquette de T.

c) Quels sont le type et la valeur de A.label ?
String - 'A'.
d) Quels sont le type et la valeur de A.enfants ?
Liste d'Arbres - [B, C, D].
e) Quels sont le type et la valeur de A.enfants[0] ?
Arbre – B.

f) Quels sont le type et la valeur de B.enfants[0].label ?
String - ’E’.
g) Quels sont le type et la valeur de A.enfants[0].enfants[1].label ?
String - ’F’.

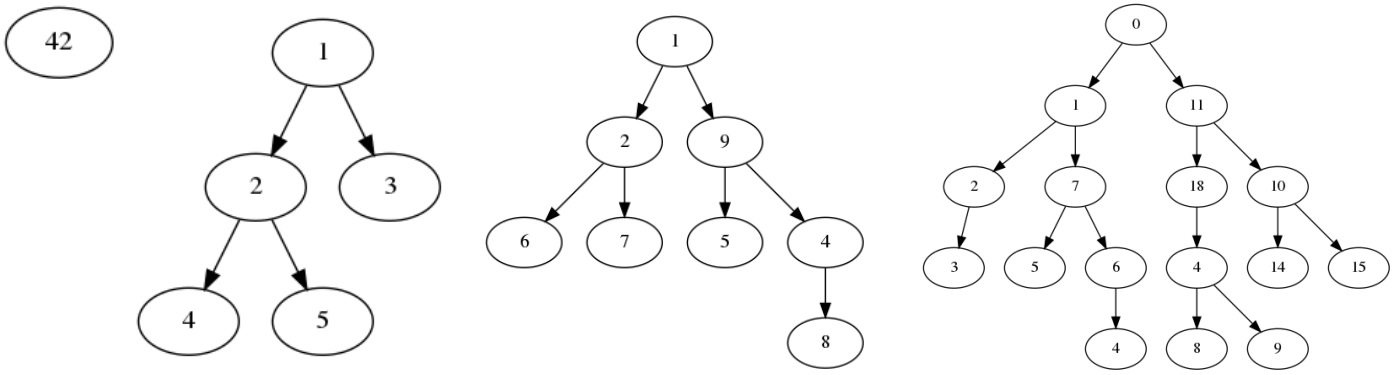
IV) RECURSIVITE (TOUJOURS SANS ORDINATEUR) :

Une fonction (ou méthode) est récursive si son code contient un appel à elle-même. Les classes arborescentes ont généralement des méthodes récursives.



Règle de non cruauté :
Tu n’ouvriras pas tes enfants pour regarder à l’intérieur. Il suffit de leur demander leur valeur, gentiment et récursivement.

Exercice 5. Un peu de lecture de code.
Pour cet exercice, on donne (dans l’ordre) les arbres a0, a1, a2 et a3 suivants :



a) Compléter le tableau ci-dessous :

	a0	a1	a2	a3
taille	1	5	8	16
hauteur	0	2	3	4
nombre de feuilles	1	3	4	7
l’étiquette la plus grande	42	5	9	18

b) Voici une première fonction qui prend un arbre en paramètre :

```
def est_une_feuille(arbre):
    return arbre.enfants == []
```

b.1) Est-ce une fonction récursive ? Non.
b.2) Préciser les valeurs des variables f1, f2, f3 et f4 suivantes : True - False - False – True.

```
f1 = est_une_feuille(a0)
f2 = est_une_feuille(a1)
f3 = est_une_feuille(a1.enfants[0])
f4 = est_une_feuille(a1.enfants[1])
```

c) Voici une autre fonction qui prend également un arbre en paramètre :

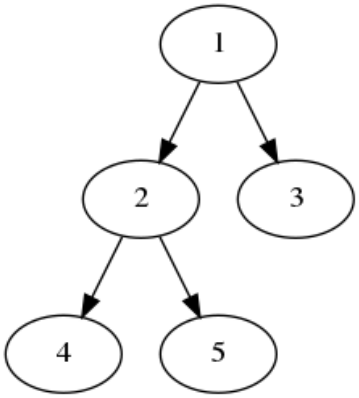
```
def mystere(arbre):
    if est_une_feuille(arbre):
        total = 1
    else:
        total = 0
        for sous_arbre in arbre.enfants:
            total= total + mystere(sous_arbre)
    return total
```

a) Est-ce une fonction récursive ? oui.

b) Quelles sont les valeurs des variables mystere(a0) , mystere(a1), mystere(a2) et mystere(a3) ?

Variables mystere(a0) = 1 , mystere(a1) = 3 , mystere(a2) = 4 et mystere(a3) = 7.

c) Voici un diagramme de fonctionnement pour a1 :

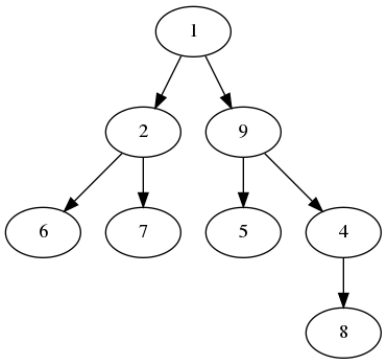



```
Condition : if est_une_feuille(arbre):
```

```
mystère(1) :
        Condition is False
        …
        mystère(2) :
                Condition is False
                …
                mystère(4):
                        Condition is True
                        return 1
                mystère(5):
                        Condition is True
                        return 1
                return total2 # ici 2
        mystere(3):
                Condition is True
                return 1

        return total1 # 2 + 1
```

Donner le diagramme de fonctionnement pour a2.



```
Condition : if est_une_feuille(arbre):
```

```
mystère(1) :
        Condition is False
        …
        mystère(2) :
                Condition is False
                …
                mystère(6):
                        Condition is True
                        return 1
                mystère(7):
                        Condition is True
                        return 1
                return total2 # ici 2 (1 + 1)
        mystere(9):
                Condition is False
                …
                mystère(5):
                        Condition is True
                        return 1
                mystère(4):
                        Condition is False
                        …
                        mystère(8):
                                Condition is True
                                return 1
                        return total8 # ici 1
                return total9 # ici 2 (1 + 1)

        return total1 # soit 4 = 2 + 2
```

d) Que fait cette fonction ? Proposer un nom plus adapté.
nombre_de_feuilles().

e) On veut écrire une fonction somme qui prend un arbre en paramètre et qui renvoie la somme des étiquettes de tous les nœuds de l'arbre.
e.1) Quelles seront les valeurs de somme(a0) et somme(a1) ?
somme(a0) = 42 et somme(a1) = 15.
e.2) Donner le diagramme de fonctionnement pour a1.

```
Condition = if est_une_feuille(arbre):
```

```
somme(1) :
        Condition is False
        …
        somme(2) :
                Condition is False
                …
                somme(4):
                        Condition is True
                        return 4
                somme(5):
                        Condition is True
                        return 5
                return total2 # ici 11(5 + 4 + 2)
        somme(3):
                Condition is True
                return 3
```

```
        return total1 # soit 15 (11 + 3 + 1)
```

e.3) Proposer un code pour cette fonction.

Version 1 :

```
def somme(arbre):
    if est_une_feuille(arbre):
        total = arbre.label
    else :
        total = arbre.label
        for sous_arbre in arbre.enfants:
            total= total + somme(sous_arbre)
    return total

assert somme(a1) == 15
assert somme(a0) == 42
```

Version refactorisée :

```
def somme(arbre):
    total = arbre.label
    for sous_arbre in arbre.enfants:
        total= total + somme(sous_arbre)
    return total

assert somme(a1) == 15
assert somme(a0) == 42
```

f) On veut écrire une fonction hauteur qui prend un arbre en paramètre et qui renvoie sa hauteur.

f.1) Quelles seront les valeurs de hauteur(a0) et hauteur(a1) ?

hauteur(a1) = 2 et hauteur(a0) = 0.

f.2) Donner le diagramme de fonctionnement pour a1.

```
Condition = if est_une_feuille(arbre):
```

```
hauteur(1) :
        Condition is False
        …
        hauteur(2) :
                Condition is False
                …
                hauteur(4):
                        Condition is True
                        return 0
                hauteur(5):
                        Condition is True
                        return 0
                return max(hauteur(4), hauteur(5)) +1 # ici 1 (max(0,0) + 1)
        hauteur(3):
                Condition is True
                return 0

        return total1 # soit 2 (max(0,1) + 1, soit 1 + 1)
```

f.3) Proposer un code pour cette fonction.

```
def hauteur(arbre):
    h = 0
        if est_une_feuille(arbre):
            return h
    else:
        for sousarbre in arbre.enfants:
```

```
        h = max(hauteur(sousarbre),h)
    return h+1

assert hauteur(a1) == 2
assert hauteur(a0) == 0
assert hauteur(a3) == 4
assert hauteur(a2) == 3
```

Autre version

```
def hauteur(arbre):
    if est_une_feuille(arbre):
        return 0
    else :
        liste_des_hauteurs = []
        for sousarbre in arbre.enfants:
            liste_des_hauteurs.append(hauteur(sousarbre))
        return 1 + max(liste_des_hauteurs)

assert hauteur(a1) == 2
assert hauteur(a0) == 0
assert hauteur(a3) == 4
assert hauteur(a2) == 3
```